CS 324:
Modeling and Simulation

Random-Number Generation

Chapter 5: Ran. Num. Gen.

- Properties of Random Numbers.
- Generation of Pseudo-Random Numbers.
- Techniques for Generating Pseudo-Random Numbers.
- Tests for Random Numbers.

Properties of Ran. Num. (1/3)

Introduction (1/3)

- Random numbers are central in applications such as simulations, electronic games (e.g. for procedural generation), and cryptography.
- A simulation of any system or process in which there are inherently random components requires a method of generating or obtaining numbers that are random, in some sense. For example, the queueing and inventory models of Chaps. 3 and 4 required interarrival times, service times, demand sizes, etc.

Properties of Ran. Num. (1/3)

Introduction (2/3)

- A random number generator (RNG) is any mechanism that produces independent random numbers. The term independent implies that the probability of producing any given random number remains the same each time a number is produced.

Properties of Ran. Num. (1/3)

Introduction (3/3)

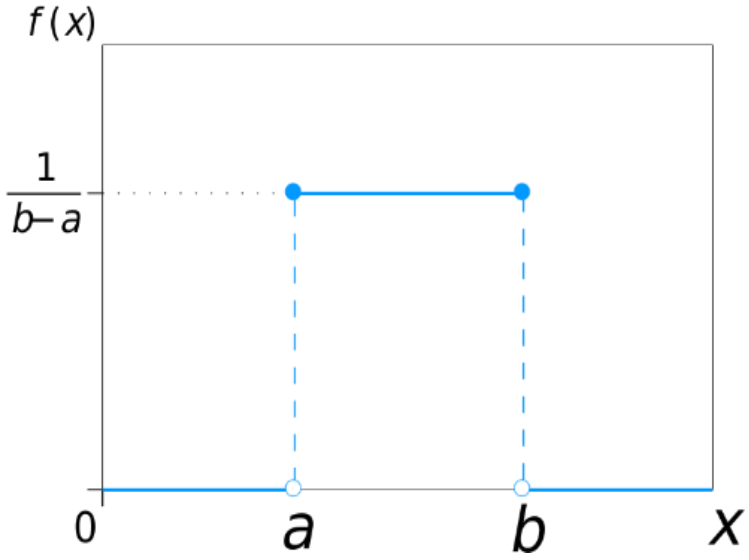
- A sequence of random numbers $R_1, R_2, \ldots,$ must have two important statistical properties:
  - Uniformity,
  - Independence.

Properties of Ran. Num. (2/3)

Uniformity (1/4)

- Random Number, R_i, must be independently drawn from a uniform distribution with probability density function (pdf):

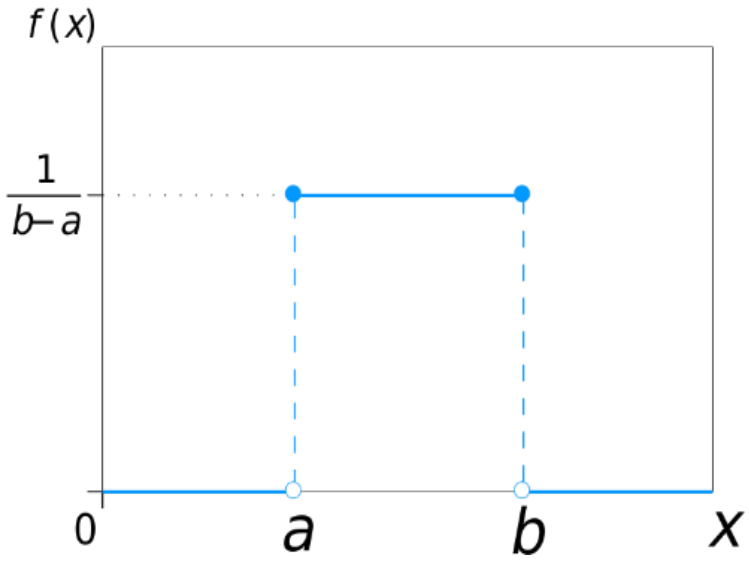
$$f(x) = \begin{cases} \dfrac{1}{b-a} & for\ x \in [a,b] \\ 0 & otherwise \end{cases}$$



Properties of Ran. Num. (2/3)

Uniformity (2/4)

$$f(x) = \begin{cases} \dfrac{1}{b-a} & for\ x \in [a,b] \\ 0 & otherwise \end{cases}$$



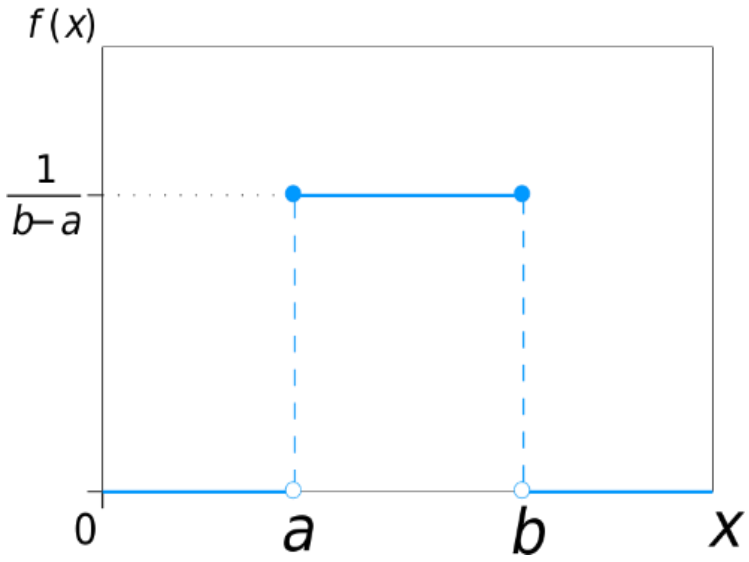
$$\text{Mean} = E(X) = \frac{a+b}{2}$$

$$\text{Variance} = V(X) = \frac{(b-a)^2}{12}$$

Properties of Ran. Num. (2/3)

Uniformity (3/4)

$$f(x) = \begin{cases} \dfrac{1}{b-a} & for\ x \in [a,b] \\ 0 & otherwise \end{cases}$$



$$\text{Mean} = E(X) = \frac{a+b}{2}$$

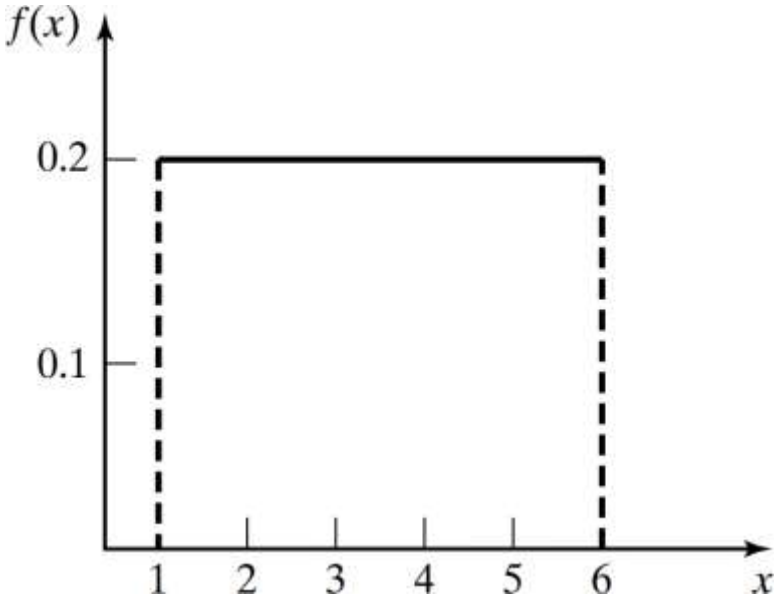
$$\text{Varaince} = V(X) = \frac{(b-a)^2}{12}$$

If we select $a = 1$ and $b = 6$

Properties of Ran. Num. (2/3)

Uniformity (3/4)

$$f(x) = \begin{cases} \frac{1}{5} & for\ x \in [1,6] \\ 0 & otherwise \end{cases}$$



$$\text{Mean} = E(X) = \frac{1+6}{2}$$

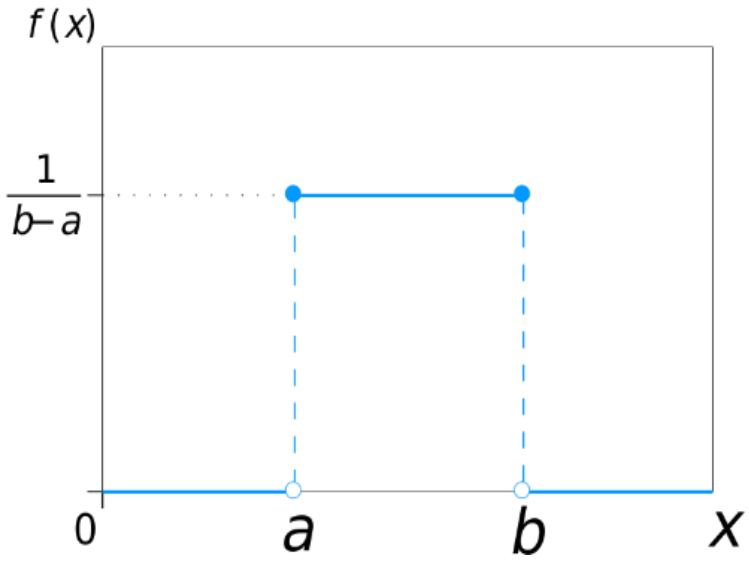
$$\text{Variance} = V(X) = \frac{(6-1)^2}{12}$$

If we select $a = 1$ and $b = 6$

Properties of Ran. Num. (2/3)

Uniformity (4/4)

$$f(x) = \begin{cases} \dfrac{1}{b-a} & for\ x \in [a,b] \\ 0 & otherwise \end{cases}$$



$$\text{Mean} = E(X) = \frac{a+b}{2}$$

$$\text{Varaince} = V(X) = \frac{(b-a)^2}{12}$$

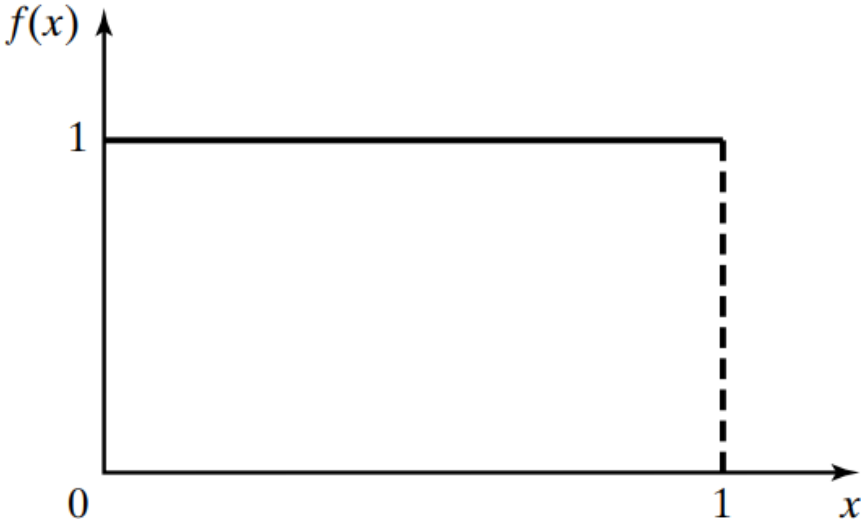
If we select $a = 0$ and $b = 1$

Standard Uniform Distribution

Properties of Ran. Num. (2/3)

Uniformity (4/4)

$$f(x) = \begin{cases} 1 & for\ x \in [0,1] \\ 0 & otherwise \end{cases}$$



$$\text{Mean} = E(X) = \frac{1}{2}$$

$$\text{Varaince} = V(X) = \frac{1}{12}$$

If we select $a = 0$ and $b = 1$

Standard Uniform Distribution

Properties of Ran. Num. (3/3)

Independence (1/2)

- In true random numbers, if you have a random number, the next random number is always *unpredictable*.
- Random processes are said to be *nondeterministic*.
- While, computers, on the other hand, are *deterministic*. Therefore, arithmetic methods are used to generate random numbers to be implemented on the computer.

Properties of Ran. Num. (3/3)

Independence (2/2)

- By definition, a true random number cannot be predicted. Numbers produced by a random number generator are calculated, and a calculated number is predictable. Thus, the numbers created by a random number generator are often referred to as *pseudorandom* numbers (PRNs).

Generation of PRNs (1/3)

Introduction

There are numerous methods that can be used to generate random values. Some important considerations concerning these methods or routines include **speed** of execution, **portability**, **replicability**, numbers produced must be statistically **independent**, **period** of the produced random sequence should be **long**, and technique used should **not require large memory** space.

Generation of PRNs (2/3)

The Routine Should Be Fast

Individual computations are inexpensive, but a simulation may require many millions of random numbers.

Portable To Different Computers

Ideally to different programming languages. This ensures the program produces same results.

Generation of PRNs (3/3)

Have Sufficiently Long Cycle

The cycle length, or period represents the length of random number sequence before previous numbers begin to repeat in an earlier order.

Replicable

Given the starting point, it should be possible to generate the same set of random numbers, completely independent of the system that is being simulated.

Techniques for PRNG (1/3)

- The Middle-Square (*midsquare*) Method. (1949)
- Linear-Congruential Generation (LCG). (1951-1958)
- Mersenne Twister (MT). (1998)
- Philox. (2011)
- 64-bit MELG. (2018)
- Squares RNG (Philox 4x32-10). (2020)

The following Wikipedia link list many algorithms for pseudo-random number generators.

https://en.wikipedia.org/wiki/List_of_random_number_generators

Techniques for PRNG (2/3)

Middle-Square (*midsquare*) Method (1/7)

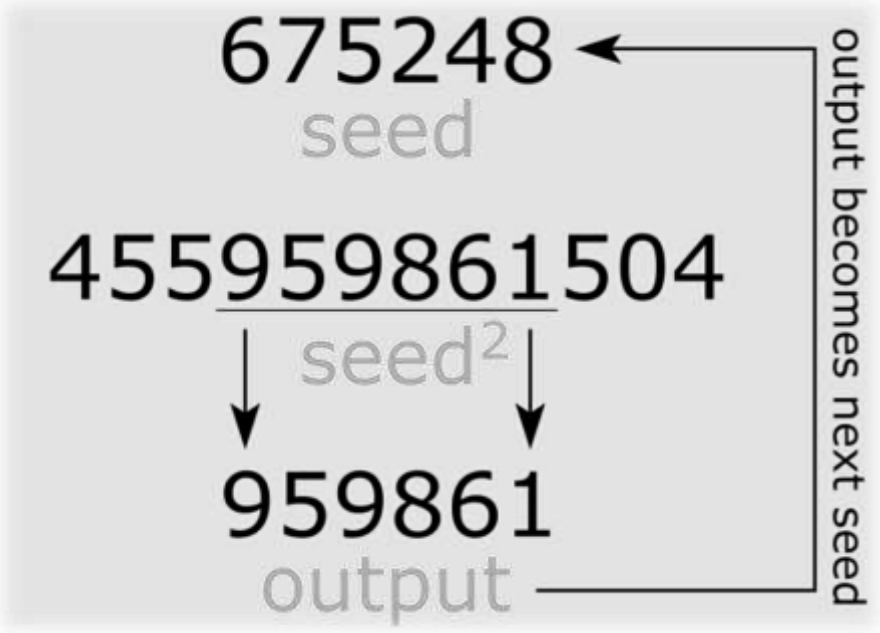
- The first such arithmetic generator, proposed in the 1949s, is the famous ***midsquare*** method.

- To generate a sequence of *n*-digit pseudorandom numbers, an *n*-digit starting value is created and squared, producing a *2n*-digit number. If the result has fewer than *2n* digits, leading zeroes are added to compensate. The middle *n* digits of the result would be the next number in the sequence and returned as the result. This process is then repeated to generate more numbers.

Techniques for PRNG (2/3)

Middle-Square (*midsquare*) Method (2/7)

- The **seed** value used to initialize an PRNG.
- Here, we used 6-digits PRN.



Techniques for PRNG (2/3)

Middle-Square (*midsquare*) Method (3/7)

- The value of *n* must be even in order for the method to work. If the value of *n* is odd then there will not necessarily be a uniquely defined 'middle *n*-digits' to select from.

Techniques for PRNG (2/3)

Middle-Square (*midsquare*) Method (4/7)

- Consider the following: If a 3-digit number is squared it can yield a 6-digit number (eg: $540^2 = 291600$).
- If there were to be a middle three digit that would leave $6 - 3 = 3$ digits to be distributed to the left and right of the middle. It is impossible to evenly distribute these digits equally on both sides of the middle number and therefore there are no 'middle digits.' It is acceptable to pad the seeds with zeros to the left in order to create an even valued *n*-digit (eg: $540 \rightarrow 0540$).

Techniques for PRNG (2/3)

Middle-Square (*midsquare*) Method (5/7)

- For a generator of *n*-digit numbers, the period can be no longer than 8^n.
- If the middle *n* digits are all zeroes, the generator then outputs zeroes forever.
- If the first half of a number in the sequence is zeroes, the subsequent numbers will be decreasing to zero.

Techniques for PRNG (2/3)

Middle-Square (*midsquare*) Method (6/7)

- The middle-squared method can also get stuck on a number other than zero. For $n = 4$, this occurs with the values 0100, 2500, 3792, and 7600.
- Other seed values form very short repeating cycles, e.g., 0540 → 2916 → 5030 → 3009. These phenomena are even more obvious when $n = 2$, as none of the 100 possible seeds generates more than 14 iterations without reverting to 0, 10, 50, 60, or a 24↔57 loop.

Techniques for PRNG (2/3)

Middle-Square (*midsquare*) Method (7/7)

- Example with seed = 7182

PRNs

i	Z_i	U_i	Z_i^2
0	7182	—	51,581,124
1	5811	0.5811	33,767,721
2	7677	0.7677	58,936,329
3	9363	0.9363	87,665,769
4	6657	0.6657	44,315,649
5	3156	0.3156	09,960,336
.	.	.	.
.	.	.	.
.	.	.	.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (1/8)

- Many random-number generators in use today are linear congruential generators (LCGs), introduced by Lehmer (1951). A sequence of integers $Z_1, Z_2, \ldots$ is defined by the recursive formula.

$$Z_i = (aZ_{i-1} + c)(\text{mod } m)$$

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (1/8)

- Many random-number generators in use today are linear congruential generators (LCGs), introduced by Lehmer (1951). A sequence of integers $Z_1, Z_2, \ldots$ is defined by the recursive formula.

$$Z_i = (aZ_{i-1} + c)(\text{mod}\, m)$$

Therefore, $0 \leq Z_i \leq m - 1$, and to obtain the desired random numbers U_i on [0, 1], we let $U_i = Z_i/m$.

modulus

is a prime number or
a power of a prime number

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (1/8)

- Many random-number generators in use today are linear congruential generators (LCGs), introduced by Lehmer (1951). A sequence of integers $Z_1, Z_2, \ldots$ is defined by the recursive formula.

$$Z_i = (aZ_{i-1} + c) \pmod{m}$$

increment

Nonnegative integer $< m$

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (1/8)

- Many random-number generators in use today are linear congruential generators (LCGs), introduced by Lehmer (1951). A sequence of integers $Z_1, Z_2, \ldots$ is defined by the recursive formula.

$$Z_i = (aZ_{i-1} + c)(\text{mod } m)$$

multiplier

Nonnegative integer $< m$

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (1/8)

- Many random-number generators in use today are linear congruential generators (LCGs), introduced by Lehmer (1951). A sequence of integers $Z_1, Z_2, \ldots$ is defined by the recursive formula.

$$Z_i = (aZ_{i-1} + c)(\text{mod } m)$$

Z_0 is the seed, nonnegative integer

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (2/8)

The LCG $Z_i = (5Z_{i-1} + 3)(\text{mod } 16)$ with $Z_0 = 7$

Example

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

m is so small; it only illustrates the arithmetic of LCGs

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (2/8)

The LCG $Z_i = (5Z_{i-1} + 3) \pmod{16}$ with $Z_0 = 7$

Example

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

$$Z_1 = (5Z_0 + 3) \bmod 16$$

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (2/8)

The LCG $Z_i = (5Z_{i-1} + 3) \pmod{16}$ with $Z_0 = 7$

Example

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

$$Z_1 = (5Z_0 + 3) \bmod 16$$

$$Z_1 = (35 + 3) \bmod 16 = 38 \bmod 16 = 6$$

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (2/8)

The LCG $Z_i = (5Z_{i-1} + 3)(\text{mod } 16)$ with $Z_0 = 7$

Example

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

$$U_1 = \frac{6}{m} = \frac{6}{16} = 0.375$$

This is the first PRN

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (2/8)

The LCG $Z_i = (5Z_{i-1} + 3) \pmod{16}$ with $Z_0 = 7$

Example

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (2/8)

The LCG $Z_i = (5Z_{i-1} + 3) \pmod{16}$ with $Z_0 = 7$

Example

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

Here, the length of a cycle is **16**

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (2/8)

The LCG $Z_i = (5Z_{i-1} + 3) \pmod{16}$ with $Z_0 = 7$

Example

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

The length of a cycle is called the ***period*** of a generator. The period is **at most *m*** called ***full period***.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (3/8)

The LCG has full period if and only if the following three conditions hold:

1. If the modulus m is ***relatively prime*** to the increment c.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (3/8)

The LCG has full period if and only if the following three conditions hold:

1. If the modulus m is ***relatively prime*** to the increment c.

The only positive integer that (exactly) divides both m and c is 1.

$\gcd(m, c) = 1$

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (3/8)

The LCG has full period if and only if the following three conditions hold:

1. If the modulus m is ***relatively prime*** to the increment c.
2. If q is a prime number that divides m, then q divides $a - 1$.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (3/8)

The LCG has full period if and only if the following three conditions hold:

1. If the modulus m is ***relatively prime*** to the increment c.
2. If q is a prime number that divides m, then q divides $a - 1$.
3. If 4 divides m, then 4 divides $a - 1$.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (4/8)

- For $c > 0$ (called ***mixed*** LCGs),
- For $c = 0$ (called ***multiplicative*** LCGs).

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (5/8)

mixed LCGs

- For a large period and high density of the U_i's on [0, 1], we want m to be large. A choice of m that is good in all these respects is $m = 2^b$, where b is the number of bits (binary digits) in a word on the computer being used that are available for actual data storage.
- For example, most computers and compilers have 32-bit words, the leftmost bit being a sign bit, so b = 31 and $m = 2^{31} > 2.1$ billion.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (6/8)

multiplicative LCGs (1/3)

- Multiplicative LCGs cannot have full period since the first condition cannot be satisfied. As we shall see, however, it is possible to obtain period $(m - 1)$ if m and a are chosen carefully. (Note: m is a prime)

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (6/8)

multiplicative LCGs (2/3)

- As with mixed generators, it's still computationally efficient to choose $m = 2^b$. However, it can be shown that in this case the period is at most 2^{b-2}, that is, $m/4$.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (6/8)

multiplicative LCGs (3/3)

- Using the multiplicative congruential method, find the period of the generator for $a = 13$, $m = 2^6 = 64$ and $X_0 = 1,2,3$ and 4.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Xi	1	13	41	21	17	29	57	37	33	45	9	53	49	61	25	5	1
Xi	2	26	18	42	34	58	50	10	2								
Xi	3	39	59	63	51	23	43	47	35	7	27	31	19	55	11	15	3
Xi	4	52	36	20	4												

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (6/8)

multiplicative LCGs (3/3)

- Using the multiplicative congruential method, find the period of the generator for $a = 13$, $m = 2^6 = 64$ and $X_0 = 1,2,3$ and 4.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Xi	1	13	41	21	17	29	57	37	33	45	9	53	49	61	25	5	1
Xi	2	26	18	42	34	58	50	10	2								
Xi	3	39	59	63	51	23	43	47	35	7	27	3					3
Xi	4	52	36	20	4												

$$\frac{m}{4} = \frac{64}{4} = 16$$

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (7/8)

Seed Selection (1/3)

- In general, the seed value used to initialize an RNG should not affect the results of the simulation. However, a wrong combination of a seed and a random generator may lead to erroneous conclusions.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (7/8)

Seed Selection (2/3)

- If the RNG has a full period and only one random variable is required, any seed value is as good as any other. However, care is required in choosing seeds for simulation requiring random numbers for more than one variable. Such types of simulation are often called ***multistream*** simulations.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (7/8)

Seed Selection (3/3)

- Most simulations are ***multistream*** simulations. For example, simulation of a single queue such as a bank with a single teller requires generating random arrival and random service times. This simulation would require two streams of random numbers: one for interarrival times and other for service times.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (8/8)

Recommendations for seed selection (1/2)

- ***Do not use zero.*** This can make certain generators produce a sequence of zeros.
- ***Do not use randomly selected seeds.***
- ***Do not use the same seed and the same random number generator*** to obtain samples of different input parameters (for example arrival times and service times) because the resulting samples may be strongly correlated.

Techniques for PRNG (3/3)

Linear Congruential Generators (LCG) (8/8)

Recommendations for seed selection (2/2)

- ***Avoid using even values as seeds***, because this results to a decrease in the period of certain random number generators.
- ***When replicating a simulation run, use different seeds***.